

Norwegian Singles Approach (NSA) Calculator

Project Report

Martin Styve Pedersen
INF221 Advanced Functional Programming @ UiB

Spring 2026

`nsa.martinstyve.no`

1 Introduction

Norwegian Singles Approach (NSA) is a web application where you can input a running race time and distance, and the application will parse the input and parse it to a VDOT number. VDOT is a generalized numerical value which represents your fitness. The web application uses the VDOT to estimate how fast you will run at other events, and suggests a pace range for running interval training, for different types of intervals. The application also has a short information card with description of running methodology.

2 Description of project

2.1 initial idea / background

My motivation behind this project is that this is a personal need I have. I am currently using `lactrace.com` but think the site has too much noise and other features. I only wanted these simple features, so I wanted to make a simpler and more focused site for this purpose.

2.2 goals of the project

A safe and reliable site for this particular need. The project should accept a race time and distance as input, compute a VDOT value, use that value to predict equivalent performances for standard race distances, and calculate interval pace ranges. It should also stay minimal, reliable, and easy to use, so that the site focuses on the specific functionality I need without unnecessary extra features.

2.3 What the project does

When a user visits the application, they are presented with a homepage with an input form. The user enters their recent race time and selects a race distance from a dropdown menu of

common race distances, or chooses to enter a custom distance. After submitting the form, the application displays a result page showing the VDOT value, equivalent race times for standard distances, and recommended pace ranges for different interval training zones. If the user enters invalid input, E.g time in wrong format or a value out of range (like a negative number), the application displays a clear error message on the homepage, and allows user to resubmit an input.

2.4 How the project turned out

The project has turned out well and matches my original goals, extended to focus more on equivalent race times. The application successfully does what I intended it to do. The simplicity of the interface and reliability of the calculations work particularly well. There is a limitation with the VDOT formula and interval pace ranges for really fast and slow times. So I have set the limits of the application to VDOT equivalent of a 3:20 1500m (WR is 3:26) and 7 hour marathon (regular cut-off time I believe) which seems reasonable.

2.5 Process

I developed this project end to end, starting with the core VDOT formula and iteratively building out parsing, safety checks, and an HTTP wrapper. I wanted to create small, working MVPs and I incrementally expanded the project and did necessary refactoring,

A technical hurdle I encountered early was with the “predictive” side of the calculator. While calculating VDOT from a time is straightforward, predicting a race time from a VDOT is mathematically complex because the formula isn’t easily reversible. I solved this by implementing a numerical inversion using a binary search to find the time that produces the target VDOT.

```
bisect :: (VDOT -> VDOT) -> VDOT -> VDOT -> VDOT -> VDOT
bisect f target low high
  | (high - low) < 0.01 = mid
  | f mid > target = bisect f target mid high
  | otherwise = bisect f target low mid
where
  mid = (low + high) / 2

equivalentTime :: VDOT -> RaceDistance -> Either VDOTError Int
equivalentTime vdot distance
  | roundedTime < minLimit = Left TimeOutOfRange
  | roundedTime > maxLimit = Left TimeOutOfRange
  | otherwise              = Right roundedTime
where
  calculatedTime = bisect ('calculateVDOT' distance) vdot 199 25201
  roundedTime    = round calculatedTime
  minLimit      = 200           -- 00:03:20
  maxLimit      = 25200        -- 07:00:00
```

As the project progressed, I replaced generic placeholders with concrete ADTs like `RunTime` and `RaceDistance`.

```
buildResultPage :: RT.RunTime -> RD.RaceDistance -> Either AppError (
  Html ())
```

And I tightened the logic with `Either` based error handling and ensured reliability through QuickCheck properties, specifically testing “round-trip” invariants.

```
prop_secondsToRunTime_Inversion :: NonNegative Int -> Property
prop_secondsToRunTime_Inversion (NonNegative totalSeconds) =
  runTimeToSec (secToRunTime totalSeconds) === totalSeconds
```

2.6 Future Extensions

One natural extension would be to add support for cycling by using functional threshold power (FTP) instead of VDOT, which would allow cyclists to use the same training guidance concept adapted to watt outputs and training zones. Other realistic improvements could include support for more workout types, better styling and visualizations, and saved user profiles for returning users.

3 How to use the project

3.1 Build and run

- clone the repository
- run `cabal build`
- run `cabal run`
- open `http://localhost:6767`
- test with `cabal test`
- prerequisites: glasgow haskell compiler (ghc)

3.2 Using the web app

The application requires a race time and a race distance as input. The time can be entered in standard formats such as `mm:ss` or `h:mm:ss`, and the distance can either be selected from a list of preset race distances or entered manually as a custom distance in meters. After submitting the form, the site calculates the VDOT value and presents equivalent race times and recommended interval paces on the result page. If the input is invalid or incomplete, the user is shown a clear error message so they can correct the values and try again.

Norwegian Singles Running Calculator

Calculate optimal training paces using the Norwegian Singles method

Time

Race distance

1500 m

Custom distance (meters)

only used when Custom distance is selected from dropdown

Analyze

Figure 1: The application homepage with input form

Norwegian Singles Running Calculator

Calculate optimal training paces using the Norwegian Singles method

expected seconds 00-59

Time

Race distance

1500 m

Custom distance (meters)

only used when Custom distance is selected from dropdown

Analyze

Figure 2: Clear error feedback for invalid time formats

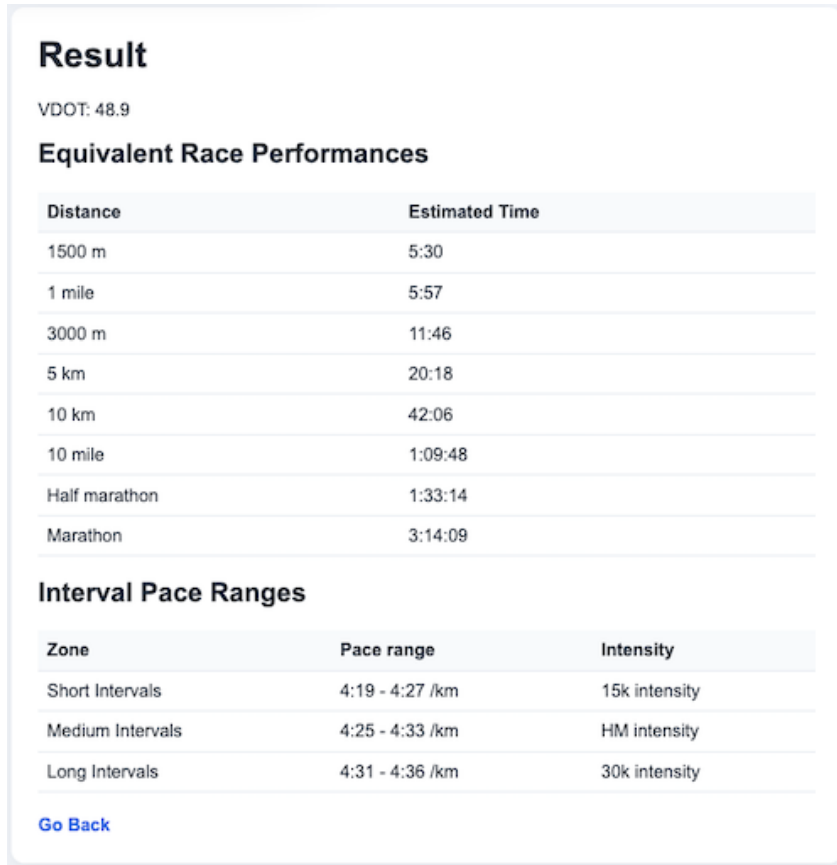


Figure 3: The result page showing calculated VDOT and training zones

4 Functional Programming Techniques Applied

4.1 Algebraic data types and domain modeling

4.1.1 Theory

Algebraic data types (ADTs) are built from sums and products. A sum type represents one of several alternatives (like `RunTime = MS | HMS`), while a product type groups related fields together. The algebraic name comes from the fact that the number of values a type can hold is literally the sum or product of its constructors, meaning `Maybe a = Nothing | Just a` can hold $1 + |a|$ values.

The key with ADTs is that by encoding domain constraints directly into types, the compiler forces you to handle all cases. You cannot accidentally create an invalid state like a distance value of -5 or a time with 75 seconds. The type system prevents it before runtime.

When you pattern match on an ADT, the compiler enforces exhaustiveness checking so that if you forget a case, the code won't compile. This is far safer than using optional string fields or "magical" values.

4.1.2 In code

```

data RunTime
  = MS Int Int      -- minutes and seconds
  | HMS Int Int Int -- hours, minutes, seconds

data RaceDistance
  = FifteenHundred
  | OneMile
  | ThreeK | FiveK | TenK
  | TenMile | HalfMarathon | Marathon
  | CustomDistance Double

```

4.1.3 How I applied it

I modeled the core domain as ADTs with `RunTime` for time entry, `RaceDistance` for distance selection, and error types (`InputError`, `VDOTError`, `AppError`) for failure cases. This meant that functions like `parseTime` naturally return `Either InputError RunTime`, making failure explicit and impossible to ignore.

4.1.4 Suitability

Very well suited. The project has well-defined domain boundaries, and ADTs map directly onto them. The only “cost” is writing extra code for constructors and pattern matches, but this is worth the correctness guarantee.

4.2 Monadic parsing combinators (Megaparsec)

4.2.1 Theory

Parser combinators let you build complex parsers from small, reusable pieces, making it far easier to handle structured input than by manually splitting text. In Haskell, parser types are compositional and work naturally with monadic sequencing (using `do` notation) and applicative operators (like `<$>` and `<|>`) to express alternatives and transformations clearly. This approach makes the input rules explicit and easy to extend, while also providing good error reporting when parsing fails.

4.2.2 In code

```

timeParser :: Parser RunTime
timeParser = do
  parts <- digits 'sepBy1' char ':'
  case parts of
    [m, s] -> if s < 60 then pure (MS m s) else customFailure
    InvalidSeconds
    [h, m, s] ->
      if m >= 60 then customFailure InvalidMinutes
      else if s >= 60 then customFailure InvalidSeconds
      else pure (HMS h m s)

```

```
_ -> customFailure InvalidFormat
```

The `do` notation hides the monadic machinery. `parts <-` runs the left parser, binds the result, and then the `case` expression uses that result to decide what to do next.

4.2.3 How I applied it

I used Megaparsec to parse time input (`mm:ss` or `h:mm:ss`) and distance selection, validating constraints (seconds must be 00–59, custom distance must be 1–50000 meters).

4.2.4 Suitability

Very suitable. The grammar is easy to read and extend. If a new time format were needed, I could add it with a new parser branch without refactoring the validation logic.

4.3 Typed error handling with `Either` and error algebra

4.3.1 Theory

`Either e a` is a sum type where `Left e` represents failure, and `Right a` represents success. The key advantage over exceptions is that the possibility of failure is encoded in the type signature.

`Either` also forms a monad where the monadic bind (`>=`) short-circuits on the first failure. Error algebra means defining domain specific error types and composing them across layers.

4.3.2 In code

```
data AppError
  = MissingRequiredInput
  | InputParseError InputError
  | VdotCalculationError VDOTError

parseTime :: Text -> Either InputError RunTime
parseTime = either (Left . bundleToInputError InvalidFormat) Right .
  runParser timeParser "" . strip

validateParams :: Maybe TimeParam -> Maybe DistanceParam -> Maybe
  CustomDistanceParam -> Either AppError (RunTime, RaceDistance)
validateParams (Just (TimeParam timeText)) (Just (DistanceParam
  distChoice)) maybeCustomDist =
  case P.parseTime timeText of
    Left err -> Left (InputParseError err)
    Right runTime -> case P.resolveDistanceSelection distChoice (
      unwrapCustomDist <$> maybeCustomDist) of
      Left err -> Left (InputParseError err)
      Right raceDistance -> Right (runTime, raceDistance)
validateParams _ _ _ = Left MissingRequiredInput
```

4.3.3 How I applied it

Every operation which can fail returns `Either`. This means the type system documents where failure can occur.

4.3.4 Suitability

Very suitable. It forces you to think about handling failures early, making the code more robust and easier to test.

4.4 Higher-order functions and partial application

4.4.1 Theory

Higher-order functions treat functions as values. They can take functions as arguments or return functions as results. Partial application is when you supply fewer arguments than a function expects, creating a new function.

4.4.2 In code

```
-- General higher-order function
calculatePaces :: VDOT -> [PaceRange]
calculatePaces vdot = map (calculateZonePace vdot) [minBound ..
    maxBound]

-- In VDOT.hs: passing a function to bisect
bisect :: (VDOT -> VDOT) -> VDOT -> VDOT -> VDOT -> VDOT
bisect f target low high
    | (high - low) < 0.01 = mid
    | f mid > target = bisect f target mid high
    | otherwise = bisect f target low mid
    where mid = (low + high) / 2
```

4.4.3 How I applied it

I used partial application to specialize generic functions. `calculateZonePace` with `vdot` fixed becomes a simpler function that only needs `Zone`. Similarly, `calculateVDOT` with the distance fixed becomes a function that `bisect` can search over.

4.4.4 Suitability

Well suited for building reusable abstractions. The code is concise and avoids the need for manual loops. By separating the logic of iteration from the domain-specific calculations, the code remains highly declarative and easier to maintain.

4.5 Recursion for numerical methods (bisection)

4.5.1 Theory

Recursion in functional code expresses repeated advancement naturally until a termination condition is met. Bisection is an algorithm that repeatedly halves an interval to find a solution. This approach is deterministic and pure.

4.5.2 In code

```
bisect :: (VDOT -> VDOT) -> VDOT -> VDOT -> VDOT -> VDOT
bisect f target low high
  | (high - low) < 0.01 = mid
  | f mid > target = bisect f target mid high
  | otherwise = bisect f target low mid
where
  mid = (low + high) / 2
```

4.5.3 How I applied it

Because the VDOT formula is not trivially invertible, I implemented recursive bisection to estimate the equivalent race time. To reinforce the domain logic, I defined `bisect` to operate specifically on the VDOT wrapper rather than generic `Double` values, as the function is only intended for these calculations.

4.5.4 Suitability

Very suitable. The termination condition is clear and controls precision.

4.6 Finite domain modeling with Enum and Bounded

4.6.1 Theory

For closed domains, deriving `Enum` and `Bounded` gives safe, total enumeration of all constructors. This lets you iterate over all cases automatically. The type system ensures you cannot accidentally miss a case.

4.6.2 In code

```
data Zone
  = ShortRep
  | MediumRep
  | LongRep
  deriving (Show, Eq, Enum, Bounded)

-- Automatically iterate over all zones
```

```
calculatePaces vdot = map (calculateZonePace vdot) [minBound ..
    maxBound]
```

4.6.3 How I applied it

I defined training zones as a closed ADT and used `[minBound .. maxBound]` to generate pace ranges.

4.6.4 Suitability

A very strong fit. Training zones are a fixed set, and using `Enum` prevents accidentally skipping or duplicating a zone.

4.7 Testing with HUnit and QuickCheck

4.7.1 Theory

Unit tests verify concrete examples, while QuickCheck asserts general laws and invariants that must hold over many randomly generated inputs. Together, these approaches provide both precision and validation.

4.7.2 In code

```
testCase "showRunTime formats sub one minute" $
    RT.showRunTime (secToRunTime 59) @?= pack "0:59"

prop_secondsToRunTime_Inversion :: NonNegative Int -> Property
prop_secondsToRunTime_Inversion (NonNegative totalSeconds) =
    runTimeToSec (secToRunTime totalSeconds) === totalSeconds
```

4.7.3 How I applied it

I used HUnit for concrete edge cases and QuickCheck for invariants. The mix ensures both specific cases and general patterns work correctly.

4.8 Conclusion Functional Programming Techniques

Overall, the techniques above complemented each other well. ADTs constrained the domain, parser combinators and `Either` made input handling robust, recursion and higher-order functions kept the core calculations modular, and mixed testing strategies validated both specific behaviors and general laws.

5 Self evaluation

5.1 What was positive about working on this project?

A positive thing about working on this project is that it gave me the chance to put the functional programming techniques from the course into practice in a real project I was personally motivated to create. It was especially useful to work with ideas from the practical sessions and lectures, like monadic parsing and QuickCheck testing.

5.2 What would I have done differently if I were to do it again?

If I were to do it again, I would probably build the frontend with plain html, css, and JavaScript instead of generating the html with Lucid, because that felt a bit cumbersome. I would also be stricter about using algebraic data types throughout the project, instead of relying on integers and text and converting them later on.